



Python Programming - 2301CS404

Lab - 12

Roll No : 453

Name : Savaliya Dhruvi B.

Exception Handling

01) WAP to handle following exceptions:

1. ZeroDivisionError
2. ValueError
3. TypeError

Note: handle them using separate except blocks and also using single except block too.

```
In [7]: # a,b = 10,0
try:
    # c = a // b
    # print(c)
    print(10 + "20")
    # c = int(input("Enter a Number:"))
except ZeroDivisionError as err:
    # print(err)
except ValueError as err:
    # print(err)
except TypeError as err:
    print(err)
```

unsupported operand type(s) for +: 'int' and 'str'

02) WAP to handle following exceptions:

1. IndexError

2. KeyError

```
In [15]: # l1 = [10,20,30]
d = {1:"one",2:"two"}
try:
    print(d[5])
except KeyError as err:
    print(err)
# try:
#     index = int(input("Enter a Index:"))
#     print(l1[index])
# except IndexError:
#     print("Index out of Bound")
# print("End of Program!")
```

5

03) WAP to handle following exceptions:

1. FileNotFoundError
2. ModuleNotFoundError

```
In [16]: # try:
#     fp = open("file1.txt","r")
# except FileNotFoundError as err:
#     print(err)
try:
    import bgdc
except ModuleNotFoundError as err:
    print(err)
```

No module named 'bgdc'

04) WAP that handles all type of exceptions in a single except block in standard error message format.

```
In [18]: name = "Dhruvi"
try:
    print(int(name))
except Exception as err:
    print(err)
```

invalid literal for int() with base 10: 'Dhruvi'

05) WAP to demonstrate else and finally block.

```
In [19]: a,b = 10,0

try:
    c = a // b
except Exception as err:
    print(err)
else:
    print("Result:",c)
finally:
    print("End of Program!")
```

integer division or modulo by zero
End of Program!

06) WAP to create an udf divide(a,b) that handles ZeroDivisionError.

```
In [20]: def divide(a,b):
        try:
            c = a // b
        except Exception as err:
            print(err)
        else:
            print("Result:",c)
        finally:
            print("End of Program!")
        divide(10,0)
```

integer division or modulo by zero
End of Program!

07) WAP to accept item prices and calculate total. Raise ValueError if price is negative, otherwise print the total bill amount.

```
In [26]: # ip2 = int(input("Enter a Item2 of Prices:"))
        total = 0;
        try:
            for i in range(0,5):
                ip = int(input("Enter a Item of Prices:"))
                if(ip < 0):
                    raise ValueError("Price can not be Negative!")
                total+=ip
        except ValueError as err:
            print(err)
        else:
            print("Total:",total)
        finally:
            print("End of Program")
```

Total: 15
End of Program

08) Create a short program that prompts the user for a list of grades separated by commas.

Split the string into individual grades and use a list comprehension to convert each string to an integer.

You should use a try statement to inform the user when the values they entered cannot be converted.

```
In [31]: try:
        s=input("Enter grade")
        li=[int(i) for i in s.split(',')]
    except Exception as err:
        print(err)
    else:
```

```

    print(li)
finally:
    print("End of Program")

```

[10, 20, 30]
End of Program

09) Accept 5 subject marks (0-100). Use assert for validation.

Sample Input (Error Case)

Enter marks: 110

Sample Output (Error Case)

AssertionError: Marks should be between 0 and 100.

Sample Input (Normal Case)

Valid marks entered

Sample Output (Normal Case)

Average Marks displayed

```

In [40]: total = 0;
try:
    for i in range(0,5):
        marks = int(input("Enter a Item of Prices:"))
        total+=marks
        assert marks < 100, "Marks should be between 0 and 100."

    avg = total / 5
except AssertionError as err:
    print(err)
else:
    print("Average Marks displayed:",avg)
finally:
    print("End of Program")
# assert marks < 0
# raise ValueError("Marks can not be Negative!")

```

Marks should be between 0 and 100.
End of Program

10) WAP that gets password from user. Password must have at least 8 characters and one digit.

Raise WeakPasswordError if invalid, print the message "Correct" otherwise.

```

In [43]: class WeakPasswordError(Exception):
pass
try:
    count = 0;
    password = input("Enter a Password.")
    for i in password:
        if i.isdigit():

```

```

        count += 1;
    if(len(password) < 8):
        raise WeakPasswordError("Password must have at least 8 characters")
    if (count < 1):
        raise WeakPasswordError("Password must have One Digit")
except WeakPasswordError as err:
    print(err)
else:
    print("Valid Password:",password)
finally:
    print("End of Program")

```

Password must have at least 8 characters
End of Program

11) WAP to raise your custom Exception named NegativeNumberError with the error message : "Cannot calculate the square root of a negative number" :

if the given number is negative.

otherwise print the square root of the given number.

```

In [46]: class NegativeNumberError(Exception):
        pass
    try:
        import math as m
        num = int(input("Enter a Number:"))
        if num < 0:
            raise NegativeNumberError("Cannot calculate the square root of a negativ
    except NegativeNumberError as err:
        print(err)
    else:
        print(m.sqrt(num))
    finally:
        print("End of Program!")

```

2.23606797749979
End of Program!

In []: ()